实验4 线程同步

- 实验目的
  - 掌握操作系统并发的概念
  - 理解并发中的生产-消费者问题
  - 熟悉Linux线程编程机制，掌握线程同步的实现方式

实验内容与要求

1. 完成Linux线程程序，完成利用Linux线程信号量及PV操作实现消费者的同步关系的程序。
2. 利用Linux线程信号量及PV操作实现哲学家就餐的同步关系的程序。

Linux多线程编程

Linux系统中主要使用进程编程，linux对线程的操作优势不如windows。

Linux系统下的多线程遵循POSIX(Portable Operating System Interface，可移植操作系统接口)线程接口，称为pthread。在Linux 内核2.6之前，进程是最主要的处理调度单元，并没支持内核线程机制，在1996年第一次获得线程的支持。

Linux多线程编程

创建线程

- 调用线程函数作为入口点
- 使用函数pthread_create()线程创建后，就开始运行相关的线程函数

所需头文件	`#include <pthread.h>`
函数原型	`int pthread_create(pthread_t *thread, pthread_attr_t *attr, void*(*start_routine)(void *), void *arg);`
函数传入值	thread：线程标志符
	attr：线程属性设置，通常取为NULL
	start_routine：线程函数的起始地址（一个以指向void的指针作为参数和返回值的函数指针）
	arg：传递给start_routine的参数
函数返回值	成功：0
	出差：返回错误码

Linux多线程编程

退出线程

- 在线程函数运行完后，自动退出
- 或使用函数pthread_exit，这是线程的主动行为

所需头文件	#include <pthread.h>
函数原型	void pthread_exit(void *retval);
函数传入值	retval：线程结束时的返回值，可由函数如pthread_join获取

Linux多线程编程

等待线程

- 由于一个进程中的多个线程是共享数据段的，通常在线程退出之后，退出线程所占用的资源并不会随着线程的终止而得到释放
- pthread_join用于将当前线程挂起来等待线程的结束，这是一个线程阻塞的函数，调用它的线程一直等待到被等待的线程结束为止
- 函数返回时，被等待线程的资源就被收回

Linux多线程编程

等待线程

所需头文件	#include <pthread.h>
函数原型	int pthread_join(pthread_t thread, void **retval);
函数传入值	thread：线程标志符，即线程ID
	retval：用户定义的指针，用来存储被等待线程的返回值(不为NULL时)
函数返回值	成功：0
	出差：返回错误码
描述	以阻塞的方式等待thread指定的线程结束。当pthread_join()函数返回时，被等待线程的资源被收回。如果线程已经结束，那么该函数会立即返回。

示例

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
void *thread_function(void *arg);
char message[] = "Hello World ";
int main() {
    int res;
    pthread_t a_thread;
    void *thread_result;
    res = pthread_create(&a_thread, NULL, thread_function, (void *)message);
    if (res != 0) {
        perror("Thread creation failed");
        exit(EXIT_FAILURE);
    }
    printf("Waiting for thread to finish...\n");
    res = pthread_join(a_thread, &thread_result);
    if (res != 0) {
        perror("Thread join failed");
        exit(EXIT_FAILURE);
    }
    printf("Thread joined, it returned %s\n", (char *)thread_result);
    printf("Message is now %s\n", message);
    exit(EXIT_SUCCESS);
}
void *thread_function(void *arg) {
    printf("thread_function is running. Argument was %s\n", (char *)arg);
    sleep(3);
    strcpy(message, "Bye!");
    pthread_exit("Thank you for the CPU time");
}
```

示例

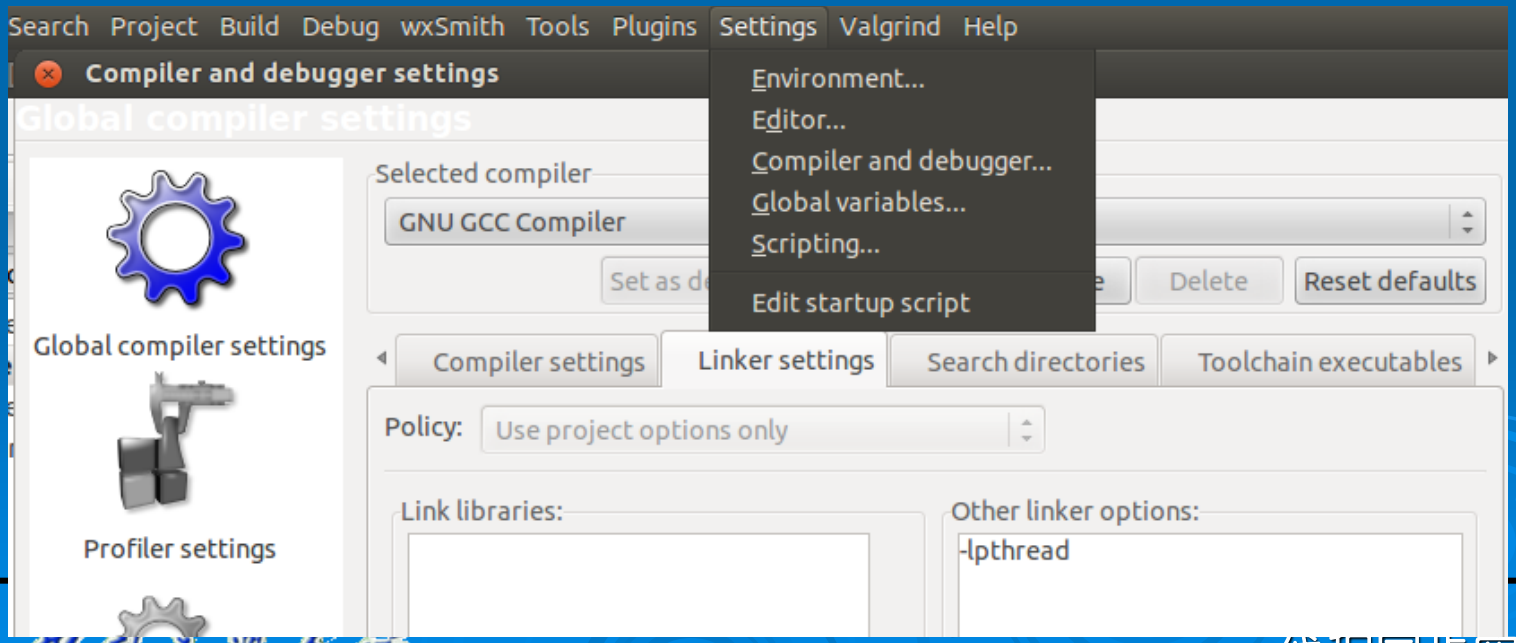
```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
void *thread_function(void *arg);
char message[] = "Hello World ";
int main() {
    int res;
    pthread_t a_thread;
    void *thread_result;
    res = pthread_create(&a_thread, NULL, thread_function, (void *)message);
    if (res != 0) {
        perror("Thread creation failed");
        exit(EXIT_FAILURE);
    }
    printf("Waiting for thread to finish...\n");
    res = pthread_join(a_thread, &thread_result);
    if (res != 0) {
        perror("Thread join failed");
        exit(EXIT_FAILURE);
    }
    printf("Thread joined, it returned %s\n", (char *)thread_result);
    printf("Message is now %s\n", message);
    exit(EXIT_SUCCESS);
}
void *thread_function(void *arg) {
    printf("thread_function is running. Argument was %s\n", (char *)arg);
    sleep(3);
    strcpy(message, "Bye!");
    pthread_exit("Thank you for the CPU time");
}
```

示例

编译：

$ gcc thread1.c -o thread1 -lpthread

编译程序包括 预编译， 编译，汇编，链接。头文件已经包含了<pthread.h>但是包含头文件仅能说明有了线程函数的声明，但是还没有实现，-lpthread是在链接阶段，链接这个库。



示例

编译：

```
$ gcc thread1.c -o thread1 -lpthread
```

运行结果：

```
Waiting for thread to finish...
thread_function is running. Argument was Hello World
Thread joined, it returned Thank you for the CPU time
Message is now Bye!
```

信号量

PV操作的含义

PV操作由P操作原语和V操作原语组成（原语是不可中断的过程），对信号量S进行操作，具体定义如下：

- 执行P操作P（S）时，信号量S的值减1，若S≥0，则继续执行；若S<0，则执行P操作的进程暂停以等待释放（即此时进程被阻塞）。
- 执行V操作V（S）时，S的值加1，若S>0，则继续执行；若S≤0，则释放一个因执行P（S）而等待的进程。

信号量

利用信号量和PV操作实现线程互斥的一般模型：

线程T1	线程T2 …	线程Tn
……	……	……
P（S）；	P（S）；	P（S）；
临界区；	临界区；	临界区；
V（S）；	V（S）；	V（S）；
……	……	…… ……

注意事项：

◆实现互斥的P、V操作必须成对出现，先做P操作，进临界区，后做V操作，出临界区。若有多个分支，要认真检查其成对性。

◆P、V操作应分别紧靠临界区的头尾部，临界区的代码应尽可能短，不能有死循环。

◆互斥信号量的初值一般为1。

Linux线程信号量

Linux多线程信号量实现了POSIX的无名信号量，主要用于线程间的互斥与同步

Linux多线程信号量用结构sem_t表示，在头文件semaphore.h中定义

所需头文件	#include <semaphore.h>
信号量声明	sem_t sem;
初始化函数（初始化信号量）	int sem_init(sem_t *sem, int pshared, unsigned int value); pshared:默认为0。value:指定信号量的初始值。 sem_init() 成功时返回 0；错误时，返回 -1，以下同。
P(S)函数	int sem_wait(sem_t * sem);
V(S)函数	int sem_post(sem_t * sem);
注销函数	int sem_destroy(sem_t * sem);

Linux线程信号量操作示例

```
int main() {
    pthread_t a_thread;
    sem_t sem;
    sem_init(&sem, 0, 1);
    ...
    pthread_create(&a_thread, NULL, thread_function, NULL);
    ...
}
void *thread_function(void *arg) {
    sem_wait(&sem);
    ...
    sem_post(&sem);
    ...
    pthread_exit(NULL);
}
```

生产消费者问题

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n= 0, e= sizeofbuffer;
void producer()
{
     while (true) {
          produce();
          semWait(e);
          semWait(s);
          append();
          semSignal(s);
          semSignal(n);
     }
}
void consumer()
{
     while (true) {
          semWait(n);
          semWait(s);
          take();
          semSignal(s);
          semSignal(e);
          consume();
     }
}
void main()
{
     parbegin (producer, consumer);
}
```

生产消费者问题

```
int main() {
    pthread_t tid_1, tid_2;
    sem_init(&p_sem, 0, TOTAL_NUM - INIT_NUM);
    sem_init(&c_sem, 0, INIT_NUM);
    sem_init(&sh_sem, 0, 1);

    pthread_create(&tid_1, NULL, productor, NULL);
    pthread_create(&tid_2, NULL, consumer, NULL);

    pthread_join(tid_1, NULL);
    pthread_join(tid_2, NULL);

    return 0;
}
```

哲学家就餐问题

```
/* program diningphilosophers */
semaphore fork[5] = {1};
semaphore room = {4};
int i;
void philosopher (int i)
{
    while (true) {
     think();
     wait (room);
     wait (fork[i]);
     wait (fork [(i+1) mod 5]);
     eat();
     signal (fork [(i+1) mod 5]);
     signal (fork[i]);
     signal (room);
    }

}
void main()
{
    parbegin (philosopher (0), philosopher (1), philosopher (2),
          philosopher (3), philosopher (4));
}
```

Figure 6.13 A Second Solution to the Dining Philosophers Problem

操作方法和实验步骤

1. 在test3.c中编写生产者和消费者两个线程函数，编译运行程序，并测试运行结果。
2. 新建工程philosophers，实现哲学家就餐问题。